



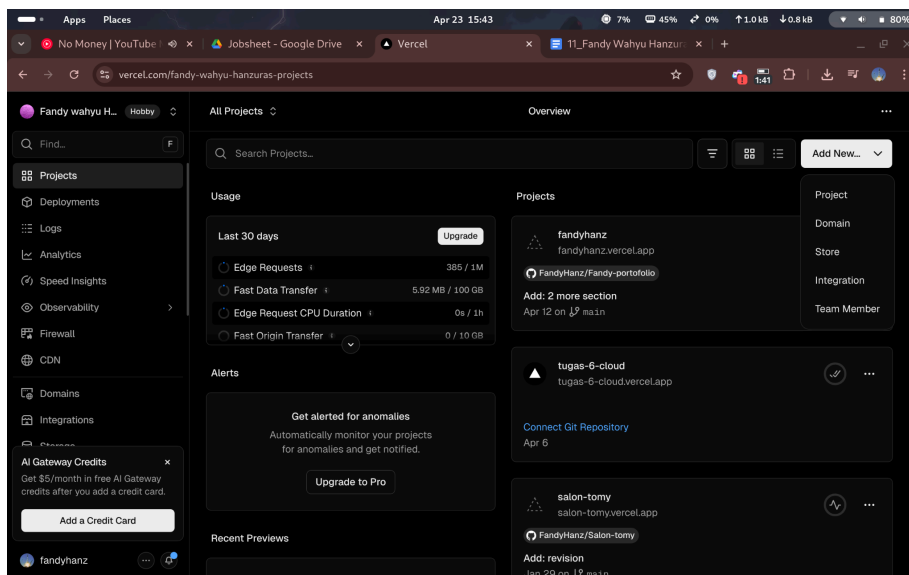
Mata Kuliah : Pemrograman Framework
Program Studi : D4 – Teknik Informatika
Semester : 6

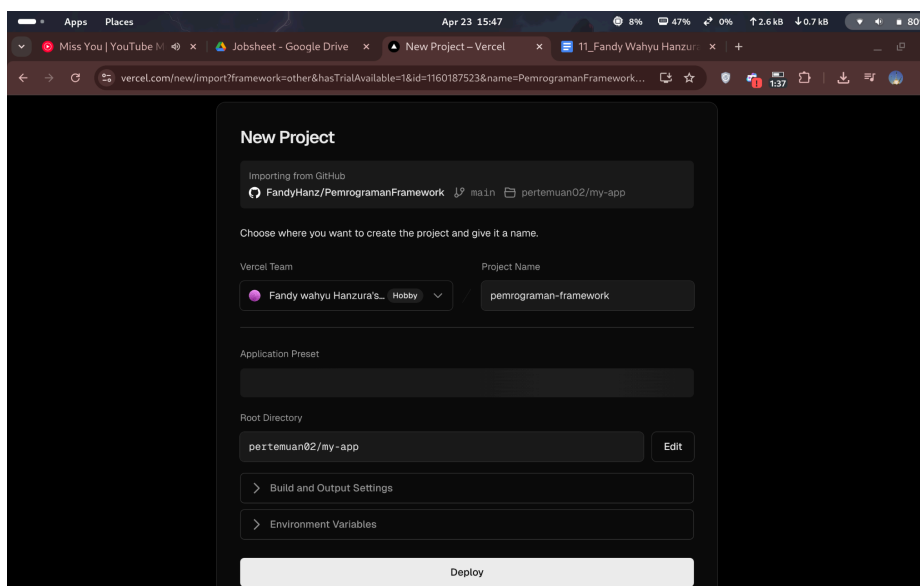
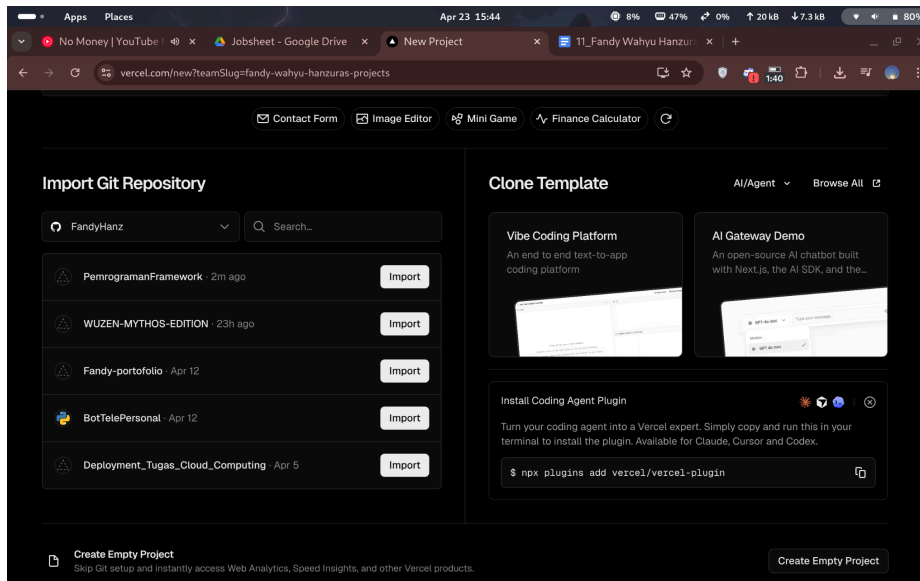
Kelas : TI-3C
NIM : 2341720165
Nama : Fandy Wahyu Hanzura
Jobsheet Ke- : 19

Laporan Jobsheet

Praktikum 1

1.





- Melakukan import pada folder proyek github



2.

```
src > pages > produk > [id].tsx > ...
7 export default function DetailProdukPage () {
14   <div>
15     <DetailProdukView products={isLoading ? [] : data.data} />
16   </div>
17 }
18
19
20 // export async function getStaticPaths() {
21 //   const res = await fetch('http://localhost:3000/api/produk');
22 //   const data = await res.json();
23 //
24 //   const paths = data.data.map((produk: any) => ({
25 //     params: { id: produk.id },
26 //   }));
27 //
28 //   return { paths, fallback: false };
29 // }
30
31
32 // export async function getStaticProps({params} : {params: {id: string}}) {
33 //   const res = await fetch('http://localhost:3000/api/produk/${params.id}');
34 //   const response : {data: ProductType} = await res.json();
35 //
36 //   return {
37 //     props: {
38 //       product: response.data
39 //     }
40 //   }
41 // }
42
```

```
src > pages > produk > [id].tsx > [id].tsx > props
4 import DetailProdukView from "@views/DetailProduk";
5 import { ProductType } from "@types/Product.types";
6
7 export default function DetailProdukPage () {
8   // const Router = useRouter();
9   // console.log(Router);
10   const { query } = useRouter();
11   const (data, error, isLoading) = useSWR(`/api/produk/${query.id}`, fetcher);
12
13   return (
14     <div>
15       <DetailProdukView products={isLoading ? [] : data.data} />
16     </div>
17   )
18 }
19
20 export async function getServerSideProps({params} : {params: {id: string}}) {
21   const res = await fetch('http://localhost:3000/api/produk/${params.id}');
22   const response : {data: ProductType} = await res.json();
23
24   return {
25     props: {
26       product: response.data
27     }
28   }
29 }
30
31 // export async function getStaticPaths() {
32 //   const res = await fetch('http://localhost:3000/api/produk');
33 //   const data = await res.json();
34 //
35 //   const paths = data.data.map((produk: any) => ({
36 //     params: { id: produk.id },
37 //   }));
38 // }
39
```

```
19 NEXT_PUBLIC_API_URL=http://localhost:3000
```

```
19
20 export async function getServerSideProps({params} : {params: {id: string}}) {
21   const res = await fetch(`${process.env.NEXT_PUBLIC_API_URL}/api/produk/${params?.id}`);
22   const response : {data: ProductType} = await res.json();
23
```



```
14
15 export async function getServerSideProps() {
16   const res = await fetch(`${process.env.NEXT_PUBLIC_API_URL}/api/produk`);
17   const response = await res.json();
18
19   return {
```

```
src > pages > produk > server.tsx > getServerSideProps > res
1 import TampilanProduk from "../views/produk";
2 import { ProductType } from "../types/Product.types";
3
4
5 export default function halamanProdukServer (props: { products: ProductType[] }) {
6   const { products } = props;
7   return(
8     <div>
9       <h1>Halaman Produk Server</h1>
10      <TampilanProduk products={products} />
11    </div>
12  )
13 }
14
15 export async function getServerSideProps() {
16   const res = await fetch(`${process.env.NEXT_PUBLIC_API_URL}/api/produk`);
17   const response = await res.json();
18
19   return {
20     props: {
21       products: response.data,
22     },
23   };
24 }
```



New Project

Importing from GitHub

FandyHanz/PemrogramanFramework main pertemuan02/my-app

Choose where you want to create the project and give it a name.

Vercel Team: Fandy wahyu Hanzura's... Hobby

Project Name: pemrograman-framework-y69x

Application Preset:

Root Directory: pertemuan02/my-app [Edit]

> Build and Output Settings

> Environment Variables

Deploy

New Project

Application Preset: Next.js

Root Directory: pertemuan02/my-app [Edit]

Build and Output Settings

Build Command: 'npm run build' or 'next build' [Toggle]

Output Directory: Next.js default [Toggle]

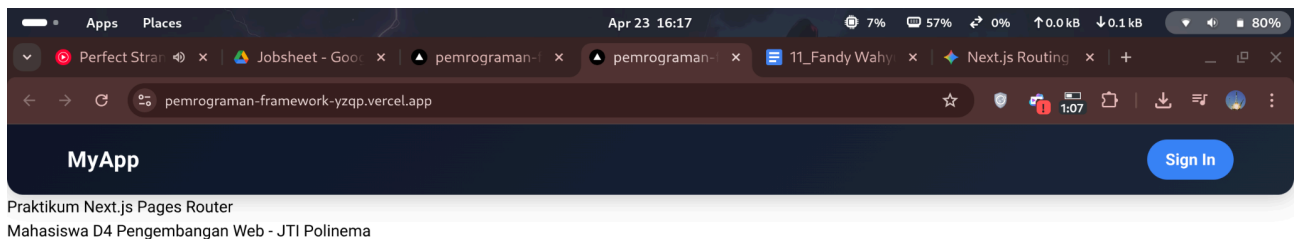
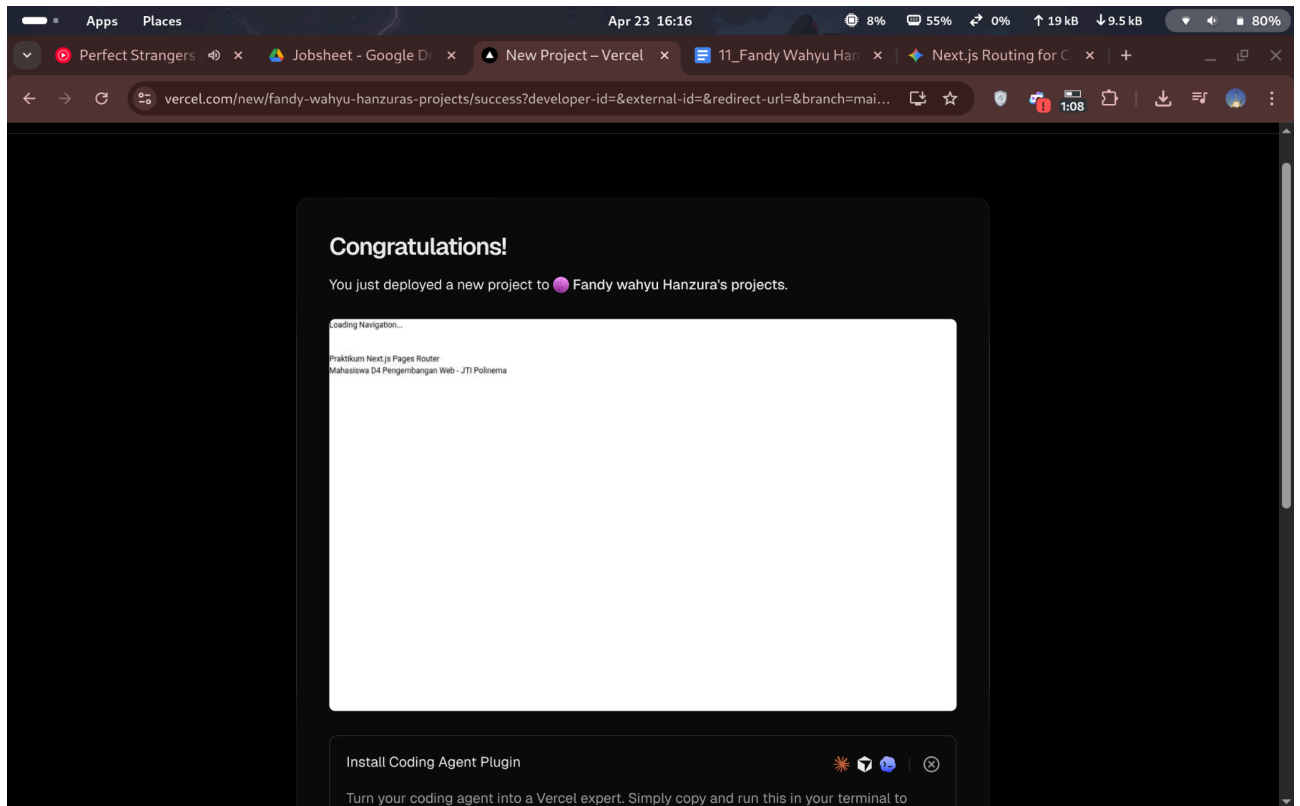
Install Command: npm install --force [Toggle]

> Environment Variables

Deploy



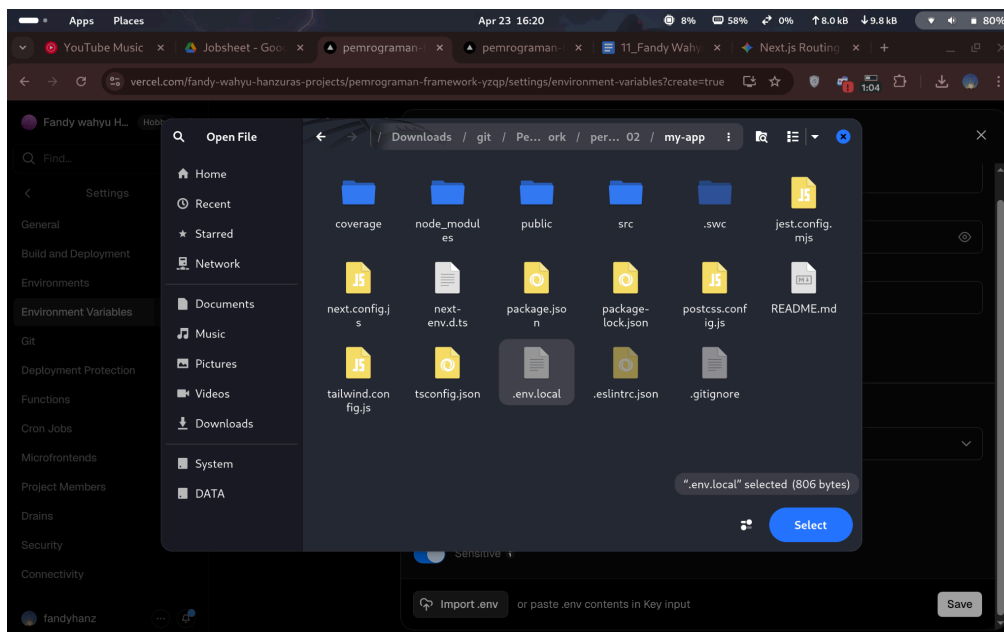
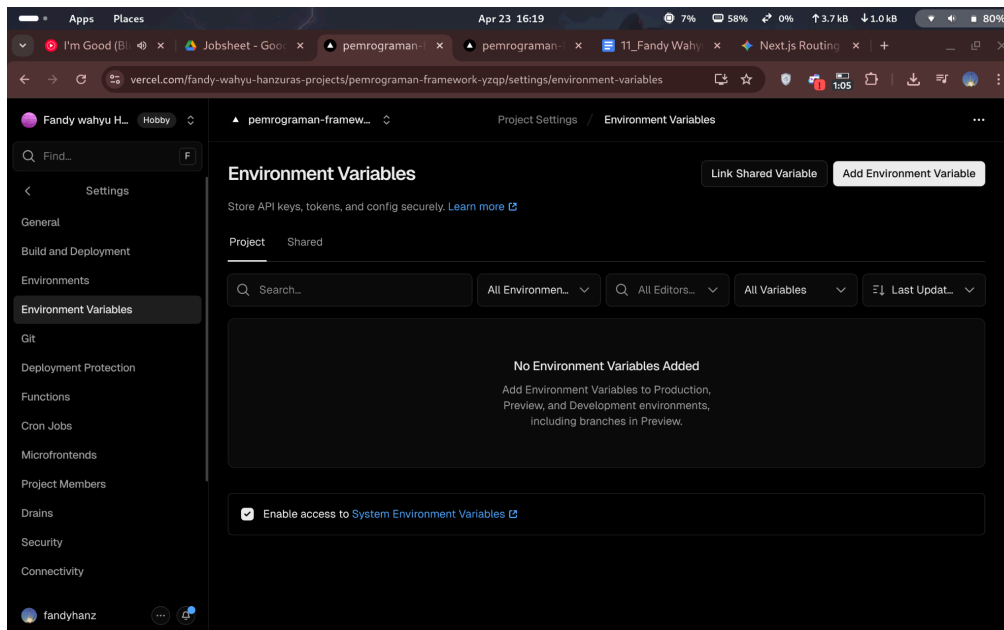
KEMENTERIAN PENDIDIKAN, KEBUDAYAAN, RISET, DAN TEKNOLOGI
POLITEKNIK NEGERI MALANG
JURUSAN TEKNOLOGI INFORMASI
Jl. Soekarno Hatta No. 9, Jatimulyo, Lowokwaru, Malang 65141
Telp. (0341) 404424 – 404425, Fax (0341) 404420
<https://jti.polinema.ac.id>

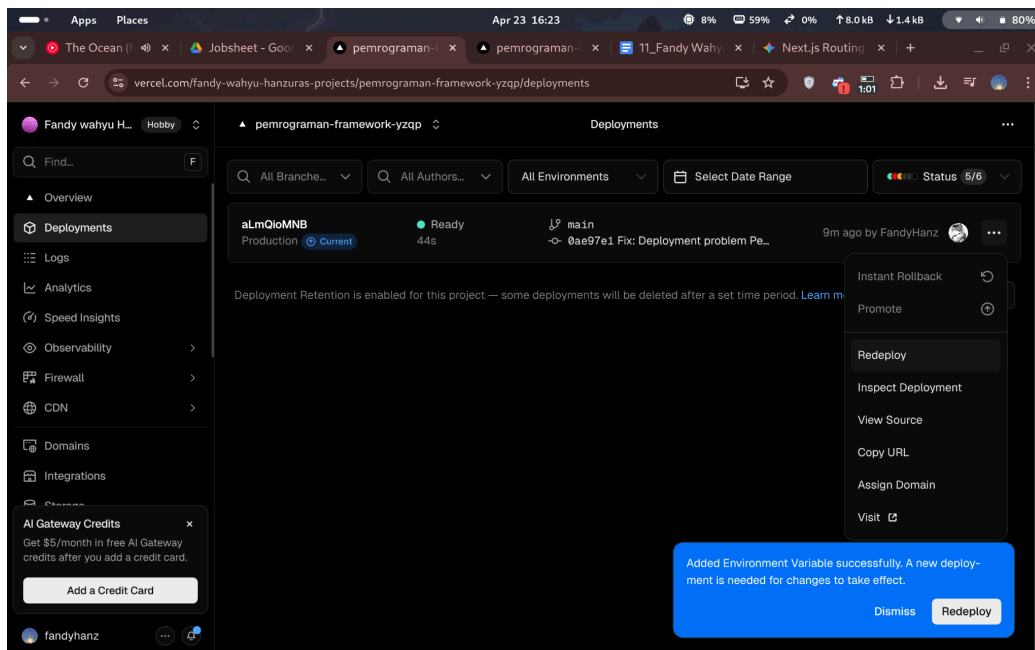
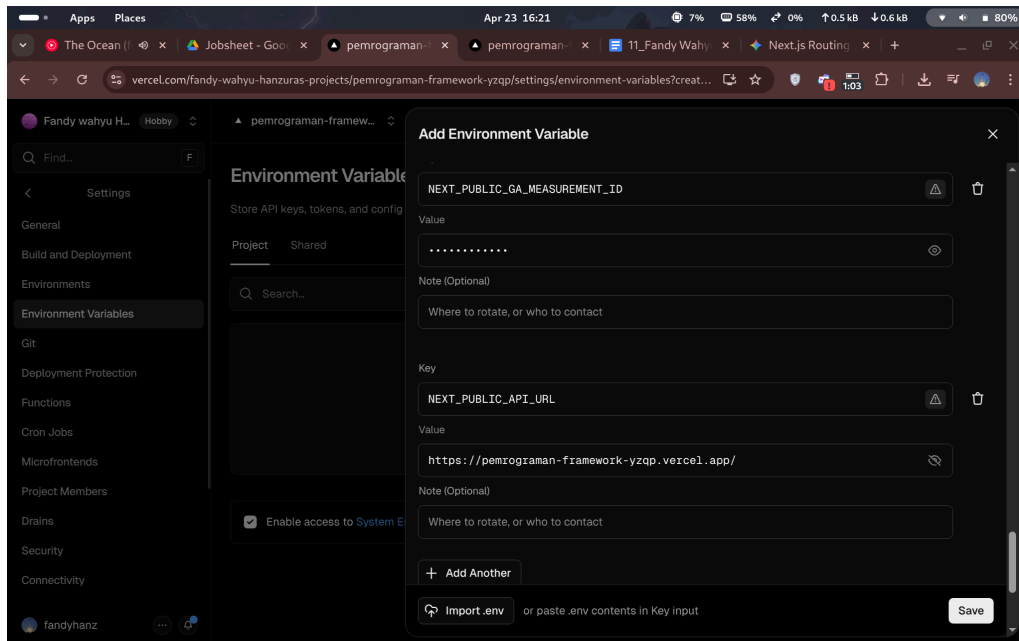


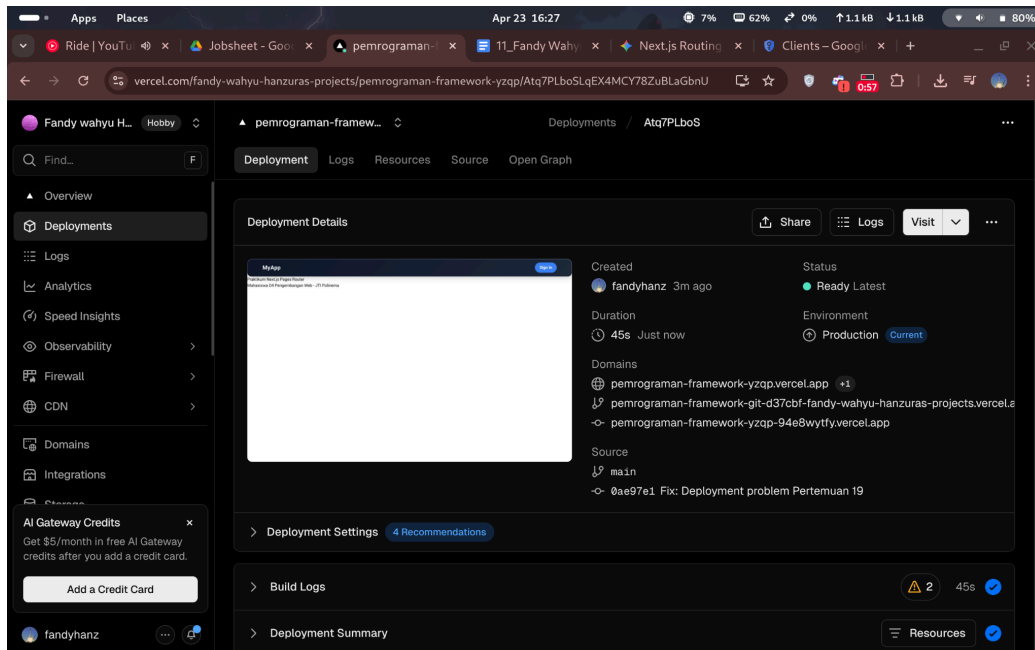


- Melakukan settings dan deployment di vercel

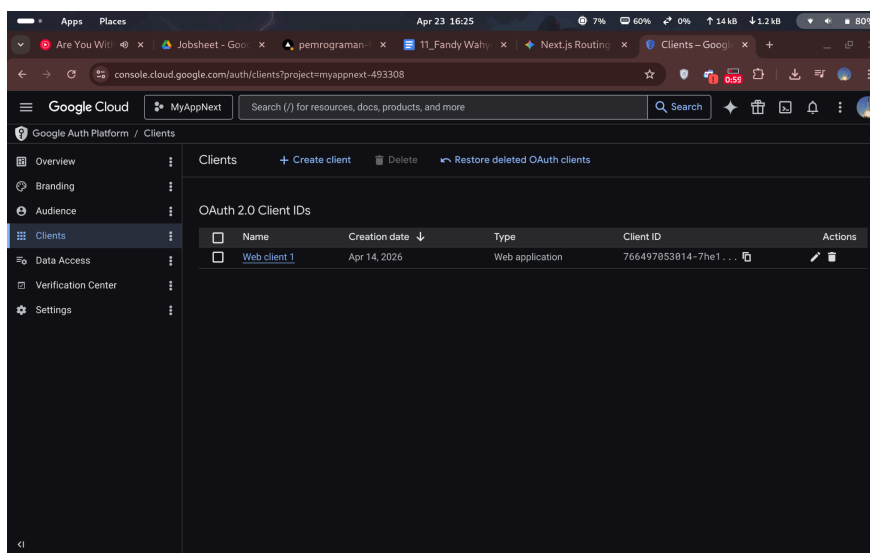
3.



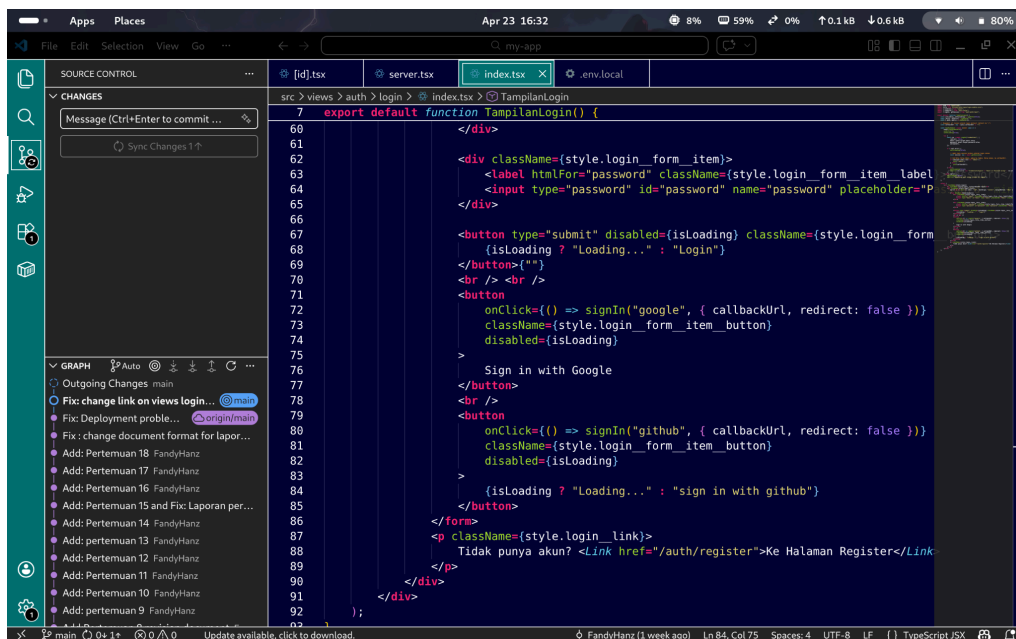
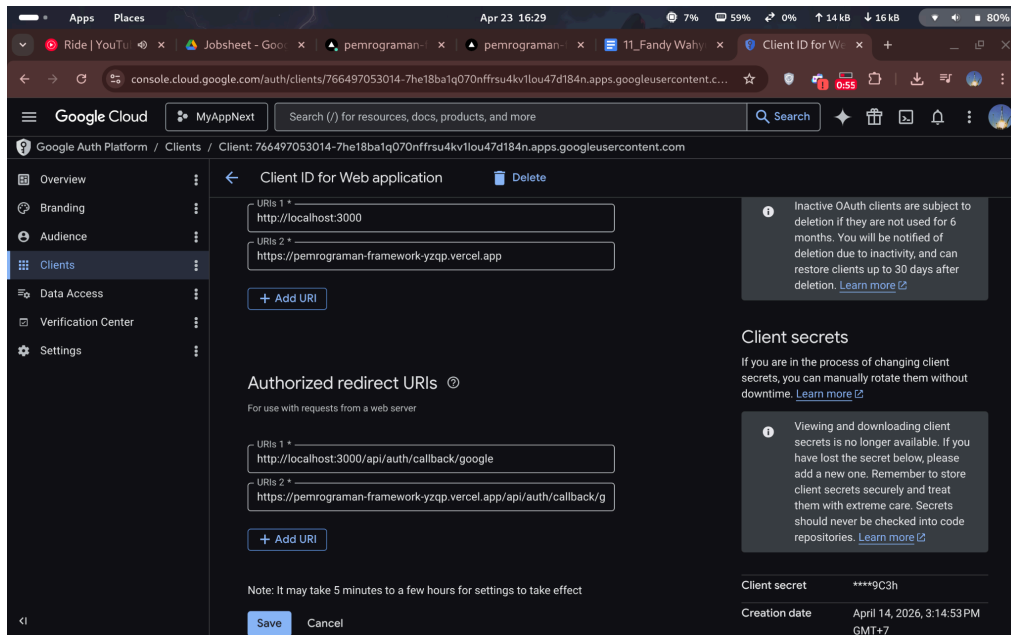


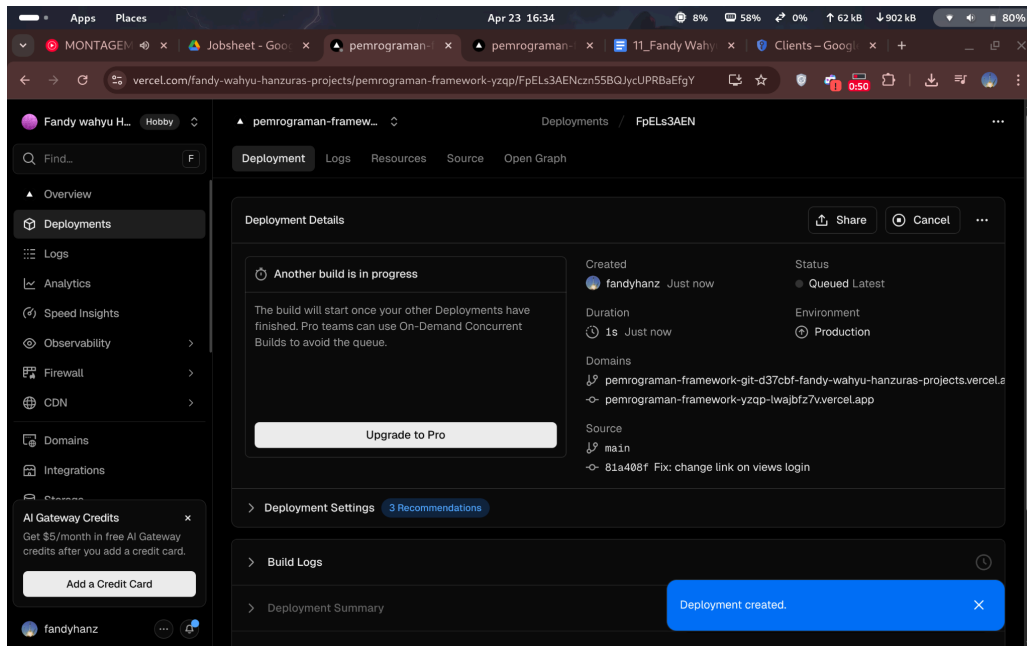


- Melakukan Penambahan environment variable dan redeploy

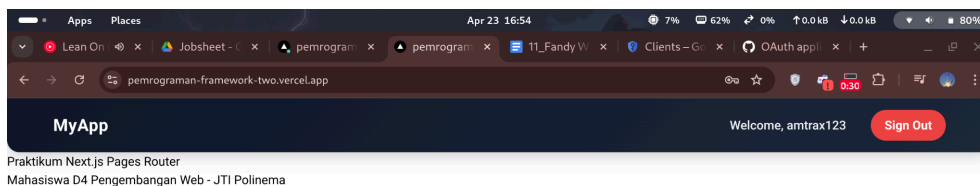


4.





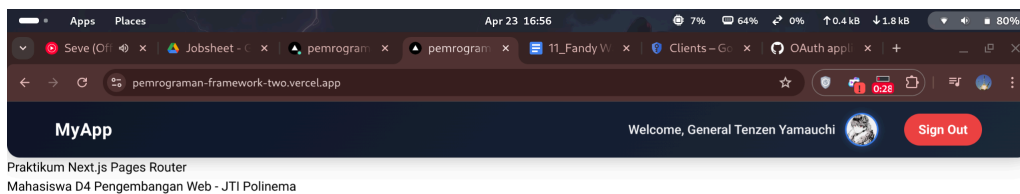
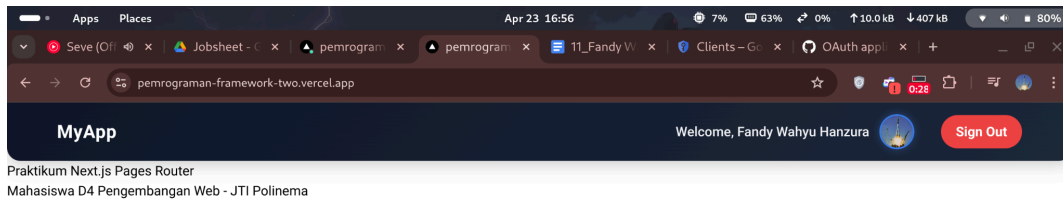
- Melakukan redeployment pada vercel



5.

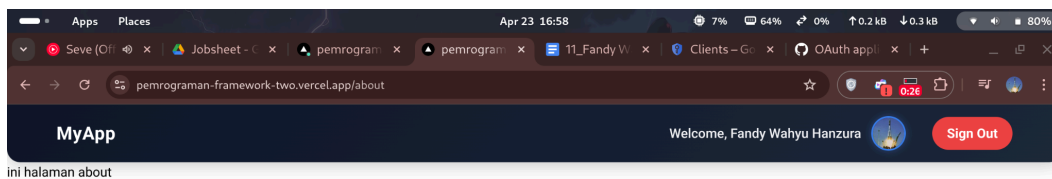
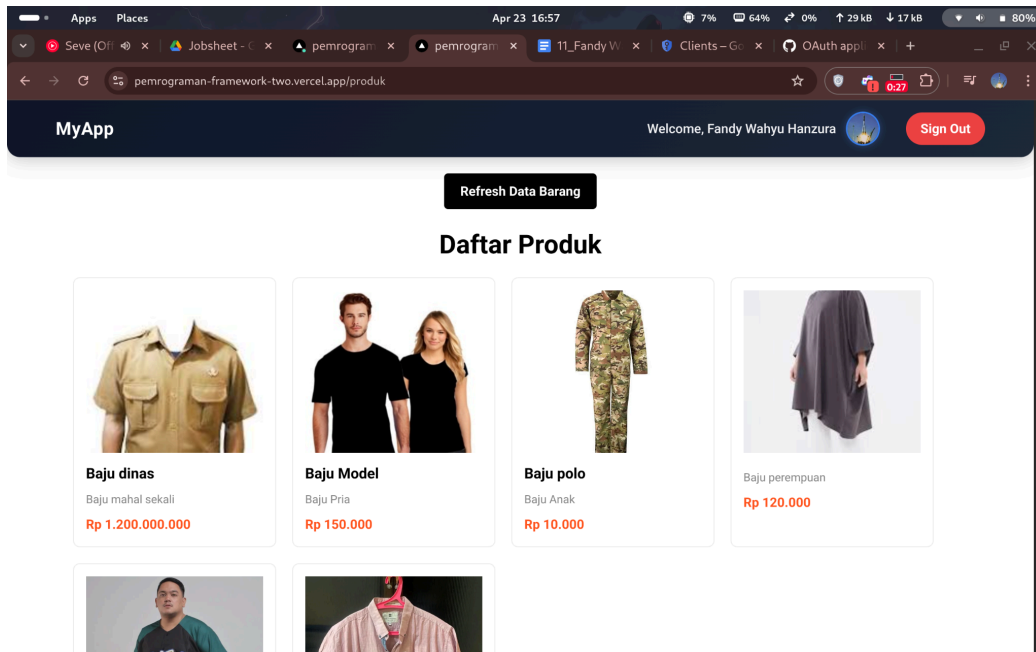


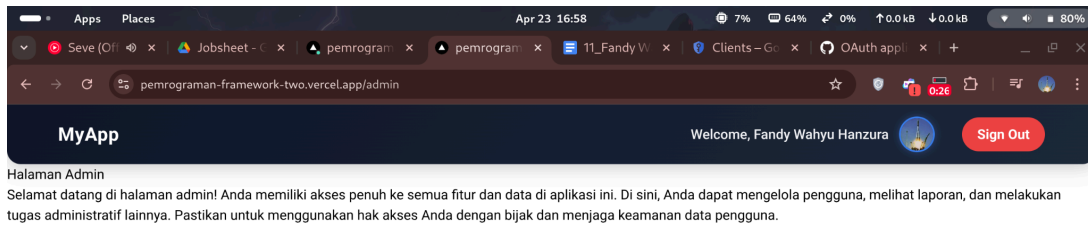
KEMENTERIAN PENDIDIKAN, KEBUDAYAAN, RISET, DAN TEKNOLOGI
POLITEKNIK NEGERI MALANG
JURUSAN TEKNOLOGI INFORMASI
Jl. Soekarno Hatta No. 9, Jatimulyo, Lowokwaru, Malang 65141
Telp. (0341) 404424 – 404425, Fax (0341) 404420
<https://jti.polinema.ac.id>





KEMENTERIAN PENDIDIKAN, KEBUDAYAAN, RISET, DAN TEKNOLOGI
POLITEKNIK NEGERI MALANG
JURUSAN TEKNOLOGI INFORMASI
Jl. Soekarno Hatta No. 9, Jatimulyo, Lowokwaru, Malang 65141
Telp. (0341) 404424 – 404425, Fax (0341) 404420
<https://jti.polinema.ac.id>





- Melakukan pengujian ulang login dengan menggunakan 3 jenis metode pada gambar pertama adalah login menggunakan credentials manual yang kedua login menggunakan google, dan menggunakan github lalu dilakukan pengecekan halaman produk, about, dan profile serta admin

Tugas Praktikum

1. Link hasil deployment: <https://pemrograman-framework-two.vercel.app/>

Pertanyaan Refleksi

1. Mengapa localhost tidak boleh digunakan di production?

localhost (atau IP **127.0.0.1**) merupakan alamat *loopback* yang merujuk pada mesin atau perangkat itu sendiri.

- **Isolasi Lingkungan:** Di lingkungan produksi, aplikasi berjalan di server *cloud*. Jika kode merujuk ke **localhost**, aplikasi akan mencoba mencari layanan (seperti database atau API) di dalam server itu sendiri, bukan di alamat publik atau server layanan terkait.



- **Aksesibilitas:** `localhost` tidak dapat diakses oleh pengguna luar melalui internet. Penggunaan URL statis `localhost` pada kode akan menyebabkan galat *Connection Refused* bagi pengguna akhir.

2. Mengapa SSG bisa gagal saat build?

SSG membangun seluruh halaman menjadi file HTML statis pada saat proses *build* berlangsung. Kegagalan biasanya disebabkan oleh:

- **Dependensi Data:** Jika sumber data (API atau CMS) tidak aktif atau mengembalikan respons galat saat perintah `next build` dijalankan, proses pembuatan halaman akan gagal.
- **Kesalahan Kodingan:** Kesalahan pada fungsi `getStaticProps` atau `getStaticPaths`, seperti mencoba mengakses variabel `window` atau `document` (yang hanya ada di browser) di lingkungan server, akan menghentikan proses *build*.
- **Limitasi Resource:** Proses *build* yang terlalu besar pada server dengan spesifikasi rendah (seperti *free tier* Vercel) dapat menyebabkan *timeout* atau kehabisan memori.

3. Apa perbedaan SSR dan SSG saat deployment?

Perbedaan utama terletak pada **kapan** HTML dihasilkan:

- **SSG (Static Site Generation):** Halaman dibuat satu kali saat proses *build*. Hasilnya adalah file statis yang siap disajikan melalui CDN. Sangat cepat, namun data tidak bersifat *real-time* kecuali dilakukan *revalidate*.
- **SSR (Server-Side Rendering):** Halaman dihasilkan di server **setiap kali ada permintaan** (*request*) dari pengguna. Ini memungkinkan data selalu *up-to-date*, namun memberikan beban lebih pada server dan waktu respons sedikit lebih lambat dibanding SSG.



4. Mengapa perlu redeploy setelah menambahkan environment?

Environment variables seringkali diinjeksikan ke dalam aplikasi pada saat proses kompilasi atau inisialisasi *runtime*.

- **Immutability:** Pada platform seperti Vercel, *build* bersifat imut (*immutable*). Nilai variabel yang baru ditambahkan di dashboard tidak akan terbaca oleh paket *build* yang lama.
- **Injeksi Sisi Klien:** Untuk variabel dengan awalan `NEXT_PUBLIC_`, nilai tersebut harus "ditanam" secara permanen ke dalam file JavaScript saat proses *build* agar dapat diakses di sisi browser pengguna.

5. Apa fungsi redirect URI pada OAuth?

Redirect URI (atau *Callback URL*) adalah parameter keamanan krusial dalam alur OAuth 2.0.

- **Validasi Keamanan:** Berfungsi sebagai mekanisme keamanan untuk memastikan bahwa kode otorisasi (*authorization code*) hanya dikirimkan kembali ke alamat aplikasi yang sah dan telah terdaftar, bukan ke situs peretas.
- **Pemulihan Alur:** Setelah pengguna berhasil login melalui penyedia layanan (seperti Google atau GitHub), penyedia tersebut memerlukan instruksi ke mana mereka harus mengarahkan kembali pengguna beserta data otentikasinya untuk diproses oleh aplikasi kita.